gen_lumenignore.py — guide

Revision: 1 **Last modified:** 2026-09-26T13:36:00Z

Source: `constitution/scripts/lumen/gen_lumenignore.py` (93 lines). Tests: `constitution/scripts/lumen/tests/test_gen_lumenignore.sh` (uses `tests/fake_ollama.py`). Citations below are `gen_lumenignore.py:<line>` unless another file is named.

Overview

Turns an ALLOW-list ("these directories are first-party") into an explicit `.lumenignore` DENY list (docstring, lines 2-9). The reason: Lumen walks the whole tree and honours only `.gitignore`, `.lumenignore`, `.gitattributes`, a built-in skip-dir set and an extension list, so a project that wants only its first-party code indexed must deny everything else (lines 4-8). The tool lists the real children of every ancestor directory of every allowed root and denies each one that is not on the path to an allowed root (lines 30-61), then appends the scope file's own `deny` patterns (lines 62-66).

It never emits a catch-all pattern such as `*`, `**` or `/*`, because Lumen refuses such roots (line 16, lines 64-65). It carries no project literal: every project-specific value comes from the scope file (line 17). Output is deterministic: sorted, de-duplicated, no timestamps, with the sha256 of the scope file in a header line (lines 13-14, 79-84).

Prerequisites

- `python3` (standard library only: `argparse`, `hashlib`, `json`, `os`, `sys`, lines 19-23).
- A repository directory to inspect and a scope file in JSON.

- To run the test: the Lumen binary (`$LUMEN_BIN` , default the newest `lumen-linux-amd64` under `~/.claude-shared/plugins/cache/claude-plugins-official/lumen/`), otherwise the test prints `SKIP` and exits 2 (`tests/test_gen_lumenignore.sh:9`).

Usage

```
gen_lumenignore.py --repo <root> --scope <scope.json> [--out <file>]
```

Argument	Meaning	Source
`--repo DIR`	Repository root whose real directory listing is read. Required	line 72
`--scope FILE`	Scope file (JSON). Required	line 73
`--out FILE`	Write the result to FILE (plain write, not atomic). Without it the result goes to stdout	lines 74, 85-89
`-h` , `--help`	argparse usage; the description is only the first docstring line	lines 71, 75

Scope file format (docstring lines 11-12):

```
{"allow": ["src_first", "dev/keep", "docs"],
 "deny": ["dev/keep/big/", "**/*.min.js"],
 "root_files": true}
```

Key	Meaning	Source
`allow`	Directories (relative to `--repo`, leading and trailing `/` stripped) that stay visible. Required non-empty	lines 27-29
`deny`	Extra gitignore patterns; a pattern not starting with `/` or `**` gets a leading `/` prepended	lines 62-66
`root_files`	Documented as "keep files directly at repo root". See Findings: it has no effect on the output	line 56

Examples

Run against a throwaway repository in a scratch directory (`src_first/` , `third/` , `dev/keep/` , `dev/drop/` , `docs/` , a root `README.md` , `dev/note.txt` and a symlink `lnk -> third`), never against the real tree.

#	Command	Status
1	`gen_lumenignore.py --help`	VERIFIED (ran; saw usage plus the one-line description, exit 0)
2	`gen_lumenignore.py --repo <r> --scope <s>` (scope: allow `src_first`, `dev/keep`, `docs`; deny `dev/keep/big/`, `**/*.min.js`, `x/y`)	VERIFIED (ran; stdout: 3 header lines `# GENERATED ...`, `# scope-sha256: <64 hex>`, `# allow: dev/keep, docs, src_first`, then `**/*.min.js`, `/dev/drop/`, `/dev/keep/big/`, `/dev/note.txt`, `/third/`, `/x/y`; exit 0)
3	Same with `--out <file>`	VERIFIED (ran; exit 0; the file was byte-identical to the stdout of example 2; nothing written in the repository)
4	Scope with `"allow": []`	VERIFIED (ran; `scope.allow is empty - refusing to emit a deny-everything file`, exit 1)
5	Scope with `"deny": ["/*"]`	VERIFIED (ran; `catch-all deny pattern refused: /*`, exit 1)
6	Scope without an `allow` key	VERIFIED (ran; same "allow is empty" refusal, exit 1)
7	Scope file that is not valid JSON, and a missing scope file	VERIFIED (ran; Python traceback ending `JSONDecodeError` / `FileNotFoundError`, exit 1 each)
8	No arguments	VERIFIED (ran; `error: the following arguments are required: --repo, --scope`, exit 2)
9	`--repo /nonexistent`	VERIFIED (ran; exit 0 and only the scope's `deny` patterns are emitted; see Edge cases)

#	Command	Status
10	`TMPDIR=<scratch> bash tests/test_gen_lume-nignore.sh`	VERIFIED (ran; `PASS: deterministic output`, `PASS: no catch-all pattern`, `PASS: control needle visible`, `PASS: lumen indexed exactly the scoped set`, `RESULT: PASS`, exit 0; it builds its own fixture repository, a fake embedding server on a random localhost port and its own `XDG_DATA_HOME` / `XDG_CONFIG_HOME` under a `mktemp` directory, and cleans up; nothing was left in the scratch `TMPDIR`)

Outputs

- The `.lumenignore` text: three header lines (`# GENERATED by gen_lumenignore.py ...`, `# scope-sha256: <sha256 of the scope file bytes>`, `# allow: <sorted allow list as written>`), then one pattern per line, sorted and de-duplicated (lines 79-84).
- Denied directories are written `/<path>/` (anchored, trailing slash); denied files inside a kept directory are written `/<path>` (lines 59, 61); scope `deny` patterns follow the rule in the table above.
- Nothing is printed on stderr in normal operation; refusals use `SystemExit(<message>)`, which prints the message on stderr and exits 1 (lines 29, 65).

Exit codes

Code	Meaning	Source
0	Text written or printed	line 92
1	Empty or missing `allow`; a catch-all deny pattern; an un-readable or invalid scope file (uncaught Python exception)	lines 29, 65, 76-77
2	Argument error (argparse)	line 75

Edge cases

- **Only the ancestor chain is walked.** Starting at the repo root, the tool lists a directory only if it is an ancestor of an allowed root (`keep` , lines 30-35, 52-54); everything else at that level is denied as a whole directory, so the deny list stays short even for a huge tree.
- **Files at the repository root are never denied** (line 56-57: when `rel == ""` and `root_files` is true they are skipped; when `root_files` is false the code falls through to `continue` without emitting anything, see Findings). Files inside a kept-but-not-allowed directory ARE denied one by one (line 58-59; example 2 shows `/dev/note.txt`).
- **Symlinks are treated as files, not directories** (`os.path.islink` , line 49). A symlink at the root is not listed; a symlink inside a kept directory would be denied as a file. Not run with a symlink inside a kept directory.
- **A missing `--repo` is not an error.** An unlistable directory is skipped (`OSError` , lines 42-45); with a nonexistent repository only the scope's `deny` patterns are emitted (example 9). The `allow` paths are not checked for existence.
- **Deny patterns are trimmed and de-duplicated**, and leading/trailing whitespace is removed before the check for catch-alls and the `/` prefixing (lines 62-66). The catch-all list is exactly `*` , `**` , `**/*` , `/*` , `/**` (line 64); other broad patterns are not refused.

- **Allowed roots that overlap** (for example `dev` and `/dev/keep/`) are accepted; leading and trailing slashes are stripped before sorting (line 27). The `# allow:` header line prints the values as written, not stripped (line 82).
- **`--out` is not atomic** (plain `open(..., "w")`, line 86); a crash mid-write leaves a truncated file.
- **The deny list is a snapshot of the working tree.** A directory added later to an ancestor is not denied until the file is regenerated; the header's `scope-sha256` covers the scope file only, not the tree.

Internal behaviour

1. Parse arguments, read the scope file bytes and parse them as JSON (lines 75-77).
2. `build`: collect the allowed roots and every ancestor path (lines 27-35); iterate a stack of directories starting at the root, listing each sorted; skip allowed children, descend into kept ancestor directories, deny everything else (lines 38-61); append the scope's `deny` patterns (lines 62-66); return the sorted unique list (line 67).
3. Prepend the three header lines and write to `--out` or stdout (lines 79-89).

Findings

1. **`root_files: false` does nothing.** The docstring describes `root_files` as "keep files directly at repo root" (line 12), but at line 56-57 only the `true` case is handled; with `false` execution reaches `continue` at line 60 without emitting a pattern for root-level files. Verified: the output body for `root_files` true and false was identical (only the `# scope-sha256:` line differed), and the root `README.md` was not listed in either.
2. **The in-source doc block is a docstring that lacks fields** the constitution's script-documentation convention lists: it has Purpose, Usage, Inputs, Outputs and Side-effects but no Exit codes, Dependencies or Cross-references (lines 2-17).
3. **`--help` shows only the first docstring line** as the description (line 71).

Related

File	Relation
lumen_verify	Verifies a Lumen index end to end; a `.lumenignore` produced here controls what it can index
`tests/test_gen_lumenignore.sh`, `tests/fake_ollama.py`	The test and the stub embedder it uses
scope_render	The CodeGraph counterpart: renders the CodeGraph scope from a scope file
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26 (source lines read on that date; examples run on a scratch repository and the test in a scratch `TMPDIR`; no real index, repository tree or `.lumenignore` of this project was touched).